

implemented directly into a CPU's floating-point circuits. You'll get the same results in C++, JavaScript, and every other language that runs on a CPU that uses IEEE 754 (which is effectively every CPU in the world).

The IEEE 754 standard, again for technical reasons beyond the scope of this book, also cannot represent all whole number values greater than 2^{53} . For example, 2^{53} and $2^{53} + 1$, as float values, both round to

9007199254740992.0:

```
>>> float(2**53) == float(2**53) + 1
True
```

As long as you use the floating-point data type, there's no workaround for these rounding errors. But don't worry. Unless you're writing software for a bank, a nuclear reactor, or a bank's nuclear reactor, rounding errors are small enough that they'll likely not be an important issue for your program. Often, you can resolve them by using integers with smaller denominations: for example, 133 cents instead of 1.33 dollars or 200 milliseconds instead of 0.2 seconds. This way, $10 + 10 + 10$ adds up to 30 cents or milliseconds rather than $0.1 + 0.1 + 0.1$ adding up to 0.30000000000000004 dollars or seconds.

But if you need exact precision, say for scientific or financial calculations, use Python's built-in `decimal` module, which is documented at <https://docs.python.org/3/library/decimal.html>. Although they're slower, `Decimal` objects are precise replacements for float values. For example, `decimal.Decimal('0.1')` creates an object that represents the exact number 0.1 without the imprecision that a 0.1 float value would have.

Passing the float value 0.1 to `decimal.Decimal()` creates a `Decimal` object that has the same imprecision as a float value, which is why the resulting `Decimal` object isn't exactly `Decimal('0.1')`. Instead, pass a string of the float value to `decimal.Decimal()`. To illustrate this point, enter the following into the interactive shell:

```
>>> import decimal
>>> d = decimal.Decimal(0.1)
>>> d
Decimal('0.1000000000000000055511151231257827021181583404541015625')
>>> d = decimal.Decimal('0.1')
>>> d
```
